

CODEX GPT-5.4 VS LOCAL LLM EVALUATION (V2)

Generated: 2026-05-26 08:25:25 UTC

GOAL

This report compares the saved local-LLM V2 answer run in docs/evaluations/answers/eval_v2_20260424T140027Z.json against a direct source/doc reading pass over data/raw/ippl. The Codex side of the comparison reuses the same source-backed reference answers and verdicts established in the first evaluation round for the overlapping question IDs.

TEST ENVIRONMENT

CODEX SIDE

- Model: GPT-5.4 (Codex)
- Method: direct source/doc reading of data/raw/ippl
- Question set: docs/evaluations/eval_questions_v2.json
- Retrieval used for grading: none; this was a source-backed comparison pass

LOCAL LLM SIDE

- Host: merlin-g-100.psi.ch
- Job / partition: 352660 on gwendolen
- Answer model: qwen2.5-coder:32b-instruct-q4_K_M
- Chunk explanation model: qwen2.5-coder:14b
- File / module / call-chain models: qwen2.5-coder:32b-instruct-q4_K_M, qwen2.5-coder:32b-instruct-q4_K_M, qwen2.5-coder:32b-instruct-q4_K_M
- Parser: tree_sitter
- Question count: 100
- Answer prompt mode: retrieval_answer_v2
- Mean answer latency: 18.51s
- Median answer latency: 17.16s

RETRIEVAL CONFIGURATION USED BY THE SAVED RUN

- Candidate k: 20
- Supplementary k: 3
- Supplementary candidate k: 10
- Vector store chunk count: 8013

IMPORTANT CAVEAT

The saved answer file and embedded vector-store manifest are consistent enough that this report can be read as a straightforward evaluation of the saved V2 run.

OVERALL RESULT

Metric	Value
--- ---:	
Questions	100
Correct	44
Partial	24
Incorrect	32
Average score (Correct=1, Partial=0.5, Incorrect=0)	0.560

MAIN FINDINGS

- Strongest areas: File Purpose, Class Responsibility. These questions were usually answerable from a single clear header, implementation file, or doc page.
- Weakest areas: Build And Install, Data Flow. These still tend to require cross-file synthesis, exact implementation location, or a sharper retrieval handoff between related subsystems.

- Compared with the first round, this V2 set removes test- and miniapp-dependent questions, so the remaining misses are a cleaner view of retrieval and answer quality over src/ plus repository docs.
- The recurring failure mode is still undercoverage: when the right implementation file or doc page is not surfaced, the local answer either abstains or answers from a nearby but incomplete chunk.
- The strongest answers remain the structural ones: file purpose, class responsibility, and direct API usage tend to do well when one primary file dominates the evidence.

CATEGORY BREAKDOWN

Category	Questions	Correct	Partial	Incorrect	Avg score
File Purpose	13	12	0	1	0.923
Definition Location	15	4	3	8	0.367
Class Responsibility	13	11	1	1	0.885
Algorithm	12	3	4	5	0.417
Data Flow	9	1	4	4	0.333
Api Usage	12	4	6	2	0.583
Parallelism And Kokkos	9	3	4	2	0.556
Boundary And Halo	5	2	1	2	0.500
Numerical Meaning	3	1	0	2	0.333
Build And Install	5	1	1	3	0.300
Testing And Workflow	4	2	0	2	0.500

DETAILED PER-QUESTION EVALUATION

FILE PURPOSE

FILE_001 - WHAT DOES IPPL.H DO?

- Verdict: Correct
- Codex reference answer: Declares the top-level IPPL runtime handles (Comm, Env, Info, Warn, Error) and the lifecycle wrappers initialize, finalize, fence, and abort.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Ippl.h
- Local answer excerpt: Answer: Ippl.h serves as a central interface header file for the IPPL library. It includes several other headers necessary for its functionalities and defines essential communication objects such as Comm and Env, along with logging o...

FILE_002 - WHAT IS THE PURPOSE OF IPPL.CPP?

- Verdict: Correct
- Codex reference answer: Implements runtime setup/teardown: command-line option parsing, MPI communicator/environment creation, logger setup, Kokkos init/finalize, and abort/fence helpers.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Ippl.cpp
- Local answer excerpt: Answer: Ippl.cpp contains several utility and lifecycle management functions for the IPPL library, including initialization (initialize), option checking (checkOption), numerical option parsing (getNumericalOption), finalization (...)

FILE_003 - WHAT DOES FFT.HPP IMPLEMENT?

- Verdict: Incorrect
- Codex reference answer: Implements FFTBase setup plus the FFT specialization family for complex/real, sine, cosine, and cosine-I transforms on IPPL fields via heFFTe.
- Comparison: Narrowed the whole file to one FFT<RCTransform, RealField> path and missed the rest of the specialization family.
- Primary sources: data/raw/ippl/src/FFT/FFT.hpp
- Local answer excerpt: Answer: FFT.hpp defines the base class template FFTBase and several specializations of the FFT class template for different types of transforms (e.g., CCTransform, RCTransform, SiTransform). It includes methods like transform...

FILE_004 - WHAT IS BAREFIELD.H RESPONSIBLE FOR?

- Verdict: Correct
- Codex reference answer: Defines the core distributed field container: Kokkos-backed storage, owned/allocated domains, field layout access, halo exchange hooks, and expression-style assignment.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Field/BareField.h
- Local answer excerpt: Answer: BareField.h defines the BareField class, which represents a field managing memory and metadata for multi-dimensional data arrays. It includes methods like getAllocated, getCommunicator, getDomain, getHalo, getHostMir...

FILE_005 - WHAT DOES FIELD.H ADD ON TOP OF BAREFIELD?

- Verdict: Correct
- Codex reference answer: Adds mesh awareness and boundary-condition management on top of BareField, plus volume integral/average helpers.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Field/Field.h
- Local answer excerpt: Answer: The Field class extends BareField by adding mesh and configurable boundary conditions. It includes methods such as get_mesh, getFieldBC, setFieldBC, deepCopy, getVolumeAverage, getVolumeIntegral, initialize, and...

FILE_006 - WHAT ROLE DOES HALOCELLS.H PLAY IN IPPL?

- Verdict: Correct
- Codex reference answer: Defines the halo-exchange machinery for fields: pack/unpack, boundary exchange, and periodic handling for ghost cells.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Field/HaloCells.h; data/raw/ippl/src/Field/HaloCells.hpp
- Local answer excerpt: Answer: The HaloCells.h header file defines the HaloCells template class and associated structs (assign, lhs_plus_assign, rhs_plus_assign) to handle halo exchange operations in parallel computations. It facilitates data communi...

FILE_007 - WHAT IS THE PURPOSE OF FIELDLAYOUT.H?

- Verdict: Correct
- Codex reference answer: Defines how a global NDIndex domain is partitioned across MPI ranks, including local subdomains, neighbors, and communication ranges.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/FieldLayout/FieldLayout.h
- Local answer excerpt: Answer: FieldLayout.h defines the FieldLayout class template, which manages the distribution and properties of fields across parallel domains in simulations within the IPPL library. The file includes methods for handling dimensions, bo...

FILE_008 - WHAT DOES PARTICLEBASE.H DEFINE?

- Verdict: Correct
- Codex reference answer: Defines the generic particle container base class with positions, optional IDs, registered attributes, and a particle layout that controls migration.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Particle/ParticleBase.h
- Local answer excerpt: Answer: ParticleBase.h defines ParticleBase, a base class for managing and storing particle data in IPPL. It includes methods like getAttributeNum, getLayout, getLocalNum, getTotalNum for accessing attributes, layout, local c...

FILE_009 - WHAT IS THE PURPOSE OF PARTICLESPATIALLAYOUT.H?

- Verdict: Correct
- Codex reference answer: Defines the spatial particle layout that decides ownership by region and migrates particles to the correct rank after motion.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Particle/ParticleSpatialLayout.h; data/raw/ippl/src/Particle/ParticleSpatialLayout.hpp

- Local answer excerpt: Answer: The ParticleSpatialLayout class in ParticleSpatialLayout.h manages spatial layout configurations for particles within computational regions. It handles region layouts and neighbor communications, providing methods to retrieve...

FILE_010 - WHAT DOES PARTICLEATTRIB.H IMPLEMENT?

- Verdict: Correct
- Codex reference answer: Implements the ParticleAttrib storage object: a Kokkos-view-backed particle attribute with resize, serialization, and mirror/access helpers.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Particle/ParticleAttrib.h
- Local answer excerpt: Answer: The ParticleAttrib class template is responsible for managing attributes associated with particles in simulations. It provides methods for serialization (deserialize), printing (print), memory management (realloc, resize...

FILE_011 - WHAT IS UNIFORMCARTESIAN.H FOR?

- Verdict: Correct
- Codex reference answer: Defines the uniform Cartesian mesh class with spacing, origin, cell volume, total mesh volume, and vertex-position helpers.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Meshes/UniformCartesian.h
- Local answer excerpt: Answer: UniformCartesian.h defines the UniformCartesian class template, which represents a uniform Cartesian mesh derived from Mesh<T, Dim>. This class provides methods for calculating vertex positions (getVertexPosition), cell vol...

FILE_012 - WHAT DOES ORTHOGONALRECURSIVEBISECTION.H DO?

- Verdict: Correct
- Codex reference answer: Declares the orthogonal recursive bisection domain-decomposition/load-balancing class and its cut/median/repartition helpers.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Decomposition/OrthogonalRecursiveBisection.h
- Local answer excerpt: Answer: The OrthogonalRecursiveBisection class in OrthogonalRecursiveBisection.h is designed to handle domain decomposition through orthogonal recursive bisection. This method divides a computational domain into smaller subdomains ef...

FILE_013 - WHAT IS THE PURPOSE OF CIC.H IN THE INTERPOLATION SUBSYSTEM?

- Verdict: Correct
- Codex reference answer: Declares the first-order cloud-in-cell interpolation helpers for scatter/gather between particles and grid fields.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Interpolation/CIC.h
- Local answer excerpt: Answer: The file CIC.h in the Interpolation subsystem defines essential interfaces and operations related to Cloud-in-Cell (CIC) interpolation. It declares template functions that compute weights, indices, and perform scattering of par...

DEFINITION LOCATION

LOC_001 - WHERE IS FFT INITIALIZATION IMPLEMENTED IN IPPL?

- Verdict: Incorrect
- Codex reference answer: FFT initialization lives in src/FFT/FFT.hpp, especially FFTBase construction/setup and the transform-specific constructors that build the heFFTe plans.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/src/FFT/FFT.hpp
- Local answer excerpt: Answer: FFT initialization in IPPL is implemented in the FFT<RCTransform, RealField> constructor within the FFT.hpp file. This constructor sets up the FFT operation using the Heffte library by configuring input and output layouts thr...

LOC_002 - WHERE DOES IPPL SET UP THE MPI ENVIRONMENT?

- Verdict: Partial
- Codex reference answer: IPPL sets up MPI in `src/Ippl.cpp::initialize()` by constructing `mpi::Environment` and `mpi::Communicator`; the low-level `MPI_Init` check/call is in `src/Communicate/Environment.cpp`.
- Comparison: The local answer found the low-level MPI constructor, but missed that the IPPL entry point is `ippl::initialize()` in `Ippl.cpp`.
- Primary sources: `data/raw/ippl/src/Ippl.cpp`; `data/raw/ippl/src/Communicate/Environment.cpp`
- Local answer error: `NameError: name 'exact_symbols' is not defined`

LOC_003 - WHERE IS KOKKOS INITIALIZED AND FINALIZED INSIDE IPPL?

- Verdict: Incorrect
- Codex reference answer: Kokkos is initialized and finalized directly in `src/Ippl.cpp` inside `ippl::initialize()` and `ippl::finalize()`.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: `data/raw/ippl/src/Ippl.cpp`
- Local answer error: `NameError: name 'exact_symbols' is not defined`

LOC_004 - WHERE IS THE ORTHOGONAL RECURSIVE BISECTION LOAD BALANCER IMPLEMENTED?

- Verdict: Correct
- Codex reference answer: The ORB load balancer is implemented in `src/Decomposition/OrthogonalRecursiveBisection.hpp` with declarations in the matching header.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: `data/raw/ippl/src/Decomposition/OrthogonalRecursiveBisection.hpp`
- Local answer excerpt: Answer: The orthogonal recursive bisection load balancer is implemented in the `OrthogonalRecursiveBisection.h` and `OrthogonalRecursiveBisection.hpp` files. The primary class template definition for this functionality is found in `Ortho...`

LOC_005 - WHERE ARE BOUNDARY CONDITIONS APPLIED TO A FIELD?

- Verdict: Partial
- Codex reference answer: Field BCs are attached in `Field::setFieldBC()` and actually applied through `BConds::apply()` and the concrete face classes in `BcTypes.hpp`.
- Comparison: The local answer contains real signal, but it misses an important scope boundary, file, or algorithm detail.
- Primary sources: `data/raw/ippl/src/Field/Field.h`; `data/raw/ippl/src/Field/BConds.hpp`; `data/raw/ippl/src/Field/BcTypes.hpp`
- Local answer excerpt: Answer: Boundary conditions are applied to a `Field` through the `BConds` class, which contains and manages various boundary condition objects (`BCondBase`). The `apply` method of these boundary condition objects is called on a `Field& f...`

LOC_006 - WHERE IS THE CIC (CLOUD-IN-CELL) SCATTER IMPLEMENTED?

- Verdict: Incorrect
- Codex reference answer: CIC scatter is implemented in `src/Interpolation/CIC.hpp`, primarily `scatterToPoint()` and `scatterToField()`.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: `data/raw/ippl/src/Interpolation/CIC.hpp`
- Local answer excerpt: Answer: The CIC (cloud-in-cell) scatter implementation is found in the `CIC.h` and `CIC.hpp` files within the `Interpolation` module. Specifically, the `scatterToPoint` and `scatterToField` functions are declared in `CIC.h`, and their defi...

LOC_007 - WHERE IS THE CIC GATHER IMPLEMENTED?

- Verdict: Incorrect
- Codex reference answer: CIC gather is implemented in `src/Interpolation/CIC.hpp`, primarily `gatherFromPoint()` and `gatherFromField()`.

- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/src/Interpolation/CIC.hpp
- Local answer excerpt: Answer: The CIC gather implementation is not explicitly detailed in the provided chunks. The chunks discuss methods related to CIC interpolation and gathering but do not provide a specific implementation of the CIC gather operation. They...

LOC_008 - WHERE IS HALO EXCHANGE FOR BAREFIELD IMPLEMENTED?

- Verdict: Partial
- Codex reference answer: Halo exchange for BareField is surfaced in src/Field/BareField.hpp and actually performed in src/Field/HaloCells.hpp.
- Comparison: The local answer found the right concept, but not the exact implementation split across the wrapper and helper file.
- Primary sources: data/raw/ippl/src/Field/BareField.hpp; data/raw/ippl/src/Field/HaloCells.hpp
- Local answer excerpt: Answer: Halo exchange for BareField is implemented in the accumulateHalo method defined within the BareField.hpp file. The accumulateHalo method likely utilizes the HaloCells class from HaloCells.h to manage halo cell exchange...

LOC_009 - WHERE ARE THE FFT-BASED OPEN-BOUNDARY POISSON SOLVER ROUTINES DEFINED?

- Verdict: Incorrect
- Codex reference answer: The FFT open-boundary Poisson routines are defined in src/PoissonSolvers/FFTOpenPoissonSolver.h and src/PoissonSolvers/FFTOpenPoissonSolver.hpp.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/src/PoissonSolvers/FFTOpenPoissonSolver.h; data/raw/ippl/src/PoissonSolvers/FFTOpenPoissonSolver.hpp
- Local answer excerpt: Answer: The FFT-based open-boundary Poisson solver routines are defined in the FFTOpenPoissonSolver.hpp and FFTOpenPoissonSolver.h files. The FFTOpenPoissonSolver class template is responsible for solving Poisson equations using FF...

LOC_010 - WHERE IS THE CONJUGATE GRADIENT SOLVER ALGORITHM IMPLEMENTED?

- Verdict: Incorrect
- Codex reference answer: The CG/PCG algorithm implementation is in src/LinearSolvers/PCG.h; the Poisson wrapper that instantiates it is src/PoissonSolvers/PoissonCG.h.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/src/LinearSolvers/PCG.h; data/raw/ippl/src/PoissonSolvers/PoissonCG.h
- Local answer excerpt: Answer: The conjugate gradient solver algorithm is implemented in the PCG class within the file LinearSolvers/PCG.h. The PoissonCG class in PoissonSolvers/PoissonCG.h utilizes this implementation through its solve method, which...

LOC_011 - WHERE IS THE FEM POISSON SOLVER IMPLEMENTED?

- Verdict: Correct
- Codex reference answer: The FEM Poisson solver is implemented in src/PoissonSolvers/FEMPoissonSolver.h.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/PoissonSolvers/FEMPoissonSolver.h
- Local answer excerpt: Answer: The FEM Poisson solver is implemented in the FEMPoissonSolver class, defined in the file FEMPoissonSolver.h located under the path data/raw/ippl/src/PoissonSolvers. This class inherits from Poisson<FieldLHS, FieldRHS> and...

LOC_012 - WHERE IS THE FDTD MAXWELL SOLVER BASE CLASS DEFINED?

- Verdict: Incorrect
- Codex reference answer: The FDTD Maxwell solver base class is defined in src/MaxwellSolvers/FDTDSolverBase.h with implementation in FDTDSolverBase.hpp.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the

code/docs materially.

- Primary sources: data/raw/ippl/src/MaxwellSolvers/FDTSolverBase.h; data/raw/ippl/src/MaxwellSolvers/FDTSolverBase.hpp
- Local answer excerpt: Answer: The FDTD Maxwell solver base class is not explicitly mentioned or defined in the provided retrieved context. The context discusses a Maxwell base class for solvers of Maxwell's equations but does not specify it as an FDTD-speci...

LOC_013 - WHERE IS THE LAGRANGESPACE CLASS DEFINED?

- Verdict: Correct
- Codex reference answer: The LagrangeSpace class is defined in src/FEM/LagrangeSpace.h with methods implemented in src/FEM/LagrangeSpace.hpp.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/FEM/LagrangeSpace.h; data/raw/ippl/src/FEM/LagrangeSpace.hpp
- Local answer excerpt: Answer: The LagrangeSpace class template is defined in the file data/raw/ippl/src/FEM/LagrangeSpace.h. This header file includes key abstractions such as DeviceStruct and provides essential methods for handling Lagrangian finite el...

LOC_014 - WHERE IS THE PARTICLE UPDATE ROUTINE (THAT MOVES PARTICLES ACROSS RANKS) IMPLEMENTED?

- Verdict: Correct
- Codex reference answer: Particle migration between ranks is implemented in ParticleSpatialLayout::update() in src/Particle/ParticleSpatialLayout.hpp.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Particle/ParticleSpatialLayout.hpp
- Local answer excerpt: Answer: The particle update routine that moves particles across ranks is primarily implemented in the update method of the ParticleSpatialLayout<T, Dim, Mesh, Properties...> class template in the ParticleSpatialLayout.hpp file. Thi...

LOC_015 - WHERE ARE IPPL TIMERS DEFINED?

- Verdict: Incorrect
- Codex reference answer: IPPL timers are defined by the singleton timing layer in src/Utility/IpplTimings.h/.cpp, not just the low-level Timer helper.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/src/Utility/IpplTimings.h; data/raw/ippl/src/Utility/IpplTimings.cpp
- Local answer excerpt: Answer: IPPL timers are primarily defined in Timer.h and managed within the IpplTimings class. The Timer class provides functionality for timing operations, while IpplTimings manages and reports timing information using instances...

CLASS RESPONSIBILITY

CLS_001 - WHAT IS THE MAIN RESPONSIBILITY OF THE BAREFIELD CLASS?

- Verdict: Correct
- Codex reference answer: BareField is the core distributed field storage class: it owns the data view, domain metadata, layout link, and halo operations.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Field/BareField.h
- Local answer excerpt: Answer: The BareField class represents a field, managing memory and metadata for multi-dimensional data arrays, and provides methods to initialize, access, and perform operations on the field data. It supports communication mechanisms ...

CLS_002 - WHAT IS THE MAIN RESPONSIBILITY OF THE FIELD CLASS COMPARED TO BAREFIELD?

- Verdict: Incorrect
- Codex reference answer: Field extends BareField with a mesh, boundary-condition container, and mesh-aware integral/average semantics.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.

- Primary sources: data/raw/ippl/src/Field/Field.h
- Local answer excerpt: Answer: The provided context does not specify the exact implementation details or the full scope of Field's functionality beyond its role as a template class representing numerical fields on a mesh. However, it suggests that the Field...

CLS_003 - WHAT DOES THE FIELDLAYOUT CLASS DO AND WHAT ARE ITS MAIN TEMPLATE PARAMETERS?

- Verdict: Partial
- Codex reference answer: FieldLayout<Dim> distributes a global NDIndex<Dim> over ranks and stores local domains, neighbors, and communication ranges; its only template parameter is Dim.
- Comparison: The local answer got the role right but blurred the distinction between template parameters and constructor/runtime inputs.
- Primary sources: data/raw/ippl/src/FieldLayout/FieldLayout.h
- Local answer excerpt: Answer: The FieldLayout class manages the distribution and properties of fields across parallel domains. It describes how an index space (represented by an NDIndex object) is distributed among MPI ranks, handles initial partitioning,...

CLS_004 - WHAT IS THE PURPOSE OF THE PARTICLEBASE CLASS AND HOW IS IT MEANT TO BE USED?

- Verdict: Correct
- Codex reference answer: ParticleBase is the user-extensible base container for particles; derived classes add attributes and register them so layout-driven migration can move them.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Particle/ParticleBase.h
- Local answer excerpt: Answer: The ParticleBase class serves as a base class for managing and storing particle data in simulations. It provides methods for accessing and updating particle attributes, layout, local counts, and total counts. Users are expected...

CLS_005 - WHAT DOES THE PARTICLESPATIALLAYOUT CLASS DO?

- Verdict: Correct
- Codex reference answer: ParticleSpatialLayout decides which spatial region/rank owns each particle and performs the migration/update workflow after particles move.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Particle/ParticleSpatialLayout.h;
data/raw/ippl/src/Particle/ParticleSpatialLayout.hpp
- Local answer excerpt: Answer: The ParticleSpatialLayout class manages spatial layout configurations for particles within computational regions. It handles region layouts and neighbor communications, providing methods to retrieve region layouts, fill hash st...

CLS_006 - WHAT IS THE RESPONSIBILITY OF UNIFORMCARTESIAN?

- Verdict: Correct
- Codex reference answer: UniformCartesian represents a uniform Cartesian mesh and exposes spacing/origin/volume and vertex-position geometry queries.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Meshes/UniformCartesian.h
- Local answer excerpt: Answer: The UniformCartesian class template represents a uniform Cartesian mesh and provides methods for initializing, setting/getting mesh spacing, calculating cell volumes, and retrieving vertex positions and spacings. It inherits fr...

CLS_007 - WHAT DOES THE MESH BASE CLASS DEFINE?

- Verdict: Correct
- Codex reference answer: Mesh is the abstract base interface for meshes: origin, grid size, spacing, vertex positions, and volume queries.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Meshes/Mesh.h
- Local answer excerpt: Answer: The retrieved context does not provide any information about a Mesh base class. The chunks cover FFTBase, Status, IpplInfo, _Variant_base, and other classes but do not mention a Mesh base class. Evidence: - data/raw/ippl/src/FF...

CLS_008 - WHAT IS THE RESPONSIBILITY OF THE COMMUNICATOR CLASS IN IPPL::MPI?

- Verdict: Correct
- Codex reference answer: `ippl::mpi::Communicator` wraps MPI point-to-point, collectives, tag handling, and managed communication buffers.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: `data/raw/ippl/src/Communicate/Communicator.h`
- Local answer excerpt: Answer: The `Communicator` class in `ippl::mpi` does not appear to be explicitly mentioned or detailed in the provided context. The closest related information is about the `Environment` class in `src/Communicate/Environment.h`, which mana...

CLS_009 - WHAT IS THE ROLE OF THE ENVIRONMENT CLASS IN IPPL::MPI?

- Verdict: Correct
- Codex reference answer: `ippl::mpi::Environment` manages MPI environment lifetime checks/teardown and provides an abort hook.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: `data/raw/ippl/src/Communicate/Environment.h`;
`data/raw/ippl/src/Communicate/Environment.cpp`
- Local answer excerpt: Answer: The `Environment` class manages MPI environment settings within IPPL, encapsulating initialization, finalization, and error handling functionalities. It takes command-line arguments (`argc`, `argv`) and an optional MPI communicat...

CLS_010 - WHAT DOES THE ARCHIVE CLASS DO IN THE COMMUNICATE MODULE?

- Verdict: Correct
- Codex reference answer: `Archive` serializes and deserializes Kokkos views and vector-valued views into raw byte buffers for MPI transfer.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: `data/raw/ippl/src/Communicate/Archive.h`;
`data/raw/ippl/src/Communicate/Archive.hpp`
- Local answer excerpt: Answer: The `Archive` class in the `Communicate` module serves as a central component for data serialization and deserialization within the IPPL codebase. It manages data buffers and provides methods such as `getBuffer`, `getBufferSize`,...

CLS_011 - WHAT IS THE ROLE OF THE BCONDS CLASS?

- Verdict: Correct
- Codex reference answer: `BConds` is the container of boundary-condition faces and orchestrates neighbor discovery, BC application, and ghost-to-physical assignment.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: `data/raw/ippl/src/Field/BConds.h`; `data/raw/ippl/src/Field/BConds.hpp`
- Local answer excerpt: Answer: The `BConds` class serves as a container for boundary conditions (BCs) in the field domain within the IPPL scientific C++ codebase. It manages and applies boundary conditions to fields by calling methods like `findBCNeighbors` an...

CLS_012 - WHAT IS THE ROLE OF THE BASEMANAGER CLASS?

- Verdict: Correct
- Codex reference answer: `BaseManager` provides the simulation loop skeleton: `pre_run`, `pre_step`, `advance`, `post_step`, and `run(nt)`.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: `data/raw/ippl/src/Manager/BaseManager.h`
- Local answer excerpt: Answer: The `BaseManager` class in IPPL serves as an abstract manager responsible for orchestrating key simulation steps, including setup, execution, and teardown operations. It defines methods such as `run`, `pre_run`, `pre_step`, `post...`

CLS_013 - WHAT DOES THE INDEX CLASS REPRESENT AND WHAT OPERATIONS DOES IT SUPPORT?

- Verdict: Correct
- Codex reference answer: `Index` represents a regular strided integer range/slice and supports range-style arithmetic and comparison operations used in domain descriptions.

- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Index/Index.h; data/raw/ippl/src/Index/Index.hpp
- Local answer excerpt: Answer: The retrieved context does not contain any information about an Index class or its operations within the IPPL codebase. Evidence: - data/raw/ippl/src/Communicate/Status.h : Status ? a communication status handle for non-blockin...

ALGORITHM

ALG_001 - HOW DOES THE FFT-BASED OPEN-BOUNDARY POISSON SOLVER COMPUTE THE ELECTROSTATIC POTENTIAL?

- Verdict: Incorrect
- Codex reference answer: The FFT open-boundary solver embeds/extends the RHS onto larger domains, FFTs both the density and a precomputed Green's function, multiplies in Fourier space, inverse transforms, and optionally computes gradients/Hessians.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/src/PoissonSolvers/FFTOpenPoissonSolver.h; data/raw/ippl/src/PoissonSolvers/FFTOpenPoissonSolver.hpp
- Local answer excerpt: Answer: The FFT-based open-boundary Poisson solver computes the electrostatic potential by transforming the charge density field from real to Fourier space using FFT. It then multiplies this transformed field with a precomputed Green's f...

ALG_002 - HOW DOES THE FFT-BASED PERIODIC POISSON SOLVER WORK IN IPPL?

- Verdict: Incorrect
- Codex reference answer: The periodic FFT solver FFTs the charge density, divides each Fourier mode by $|k|^2$ (with the zero mode handled separately), then inverse transforms; gradients are computed by multiplying by $ik/|k|^2$ before the inverse FFT.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/src/PoissonSolvers/FFTPeriodicPoissonSolver.hpp
- Local answer excerpt: Answer: The FFTPeriodicPoissonSolver class is designed to solve periodic Poisson equations using Fast Fourier Transform (FFT) techniques within the IPPL scientific C++ codebase. It encapsulates the FFT-based solution process, including...

ALG_003 - HOW DOES THE TRUNCATED GREEN PERIODIC POISSON SOLVER DIFFER FROM THE STANDARD PERIODIC ONE?

- Verdict: Incorrect
- Codex reference answer: The truncated-Green periodic solver replaces the simple $1/|k|^2$ kernel with the FFT of a truncated/erf-smoothed Green's function $\text{forceConstant} * \text{erf}(\alpha r) / r$, then follows the same transform-multiply-inverse pattern.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/src/PoissonSolvers/FFTTruncatedGreenPeriodicPoissonSolver.h; data/raw/ippl/src/PoissonSolvers/FFTTruncatedGreenPeriodicPoissonSolver.hpp
- Local answer excerpt: Answer: The truncated Green periodic Poisson solver differs from the standard periodic Poisson solver by including a step to compute and regularize a truncated Green's function. This is done through the greensFunction method, which is ...

ALG_004 - HOW DOES IPPL'S CG SOLVER ITERATE IN THE PCG.H IMPLEMENTATION?

- Verdict: Partial
- Codex reference answer: In PCG.h, the CG iteration forms residual/direction fields, applies the operator, computes alpha and beta from inner products, updates the solution/residual/direction, and stops on tolerance or iteration count; the preconditioned variant inserts M^{-1} applications on the residual.
- Comparison: The local answer captured the plain CG loop but did not clearly separate it from the preconditioned path.
- Primary sources: data/raw/ippl/src/LinearSolvers/PCG.h
- Local answer excerpt: Answer: IPPL's CG solver iterates through a method that is not explicitly detailed in the provided chunks. However, based on the information given, the PCG class inherits

from CG, and both classes have methods like initializeFields...

ALG_005 - HOW IS CLOUD-IN-CELL INTERPOLATION PERFORMED IN IPPL (BOTH SCATTER AND GATHER DIRECTIONS)?

- Verdict: Correct
- Codex reference answer: CIC computes lower/upper weights per axis, scatters with atomic adds to the 2^{Dim} neighboring grid points, and gathers by summing the same weighted stencil back to particles.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Interpolation/CIC.h; data/raw/ippl/src/Interpolation/CIC.hpp
- Local answer excerpt: Answer: Cloud-in-Cell (CIC) interpolation in IPPL involves specific functions for both scatter and gather operations. The scatterToPoint function in CIC.hpp handles the distribution of particle data to grid points using atomic operat...

ALG_006 - HOW DOES THE STANDARD FDTD SOLVER ADVANCE THE ELECTROMAGNETIC FIELD IN TIME?

- Verdict: Incorrect
- Codex reference answer: The standard FDTD solver advances the source-like field with a second-order finite-difference update using the previous and current time levels plus neighboring cells and source terms, then fills halos and reapplies BCs.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/src/MaxwellSolvers/StandardFDTDsSolver.hpp
- Local answer excerpt: Answer: The StandardFDTDsSolver class advances the electromagnetic field in time by overriding the step() method from its base class FDTDSolverBase. The step() method is part of the FDTD simulation loop and is responsible for upda...

ALG_007 - HOW DOES THE LAGRANGESPACE CLASS EVALUATE DEGREES OF FREEDOM PER ELEMENT?

- Verdict: Partial
- Codex reference answer: LagrangeSpace evaluates element DOFs by mapping local element indices to global DOF indices, then evaluates order-1 reference-element shape functions and gradients on those local DOFs.
- Comparison: The local answer focused on the compile-time DOF count and missed most of the actual element-DOF evaluation logic.
- Primary sources: data/raw/ippl/src/FEM/LagrangeSpace.h; data/raw/ippl/src/FEM/LagrangeSpace.hpp
- Local answer excerpt: Answer: The LagrangeSpace class uses the numElementDOFs parameter to specify the number of degrees of freedom (DOFs) per element. This parameter is passed during the instantiation of the LagrangeSpace template class and represents ...

ALG_008 - HOW IS GAUSS-JACOBI QUADRATURE IMPLEMENTED AND USED IN IPPL FEM?

- Verdict: Correct
- Codex reference answer: Gauss-Jacobi quadrature computes nodes/weights by Newton iteration on Jacobi polynomial roots, starting from Chebyshev or LehrFEM-style guesses, and uses the resulting rule on reference elements in FEM.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/FEM/Quadrature/GaussJacobiQuadrature.h; data/raw/ippl/src/FEM/Quadrature/GaussJacobiQuadrature.hpp
- Local answer excerpt: Answer: The Gauss-Jacobi quadrature is implemented through the GaussJacobiQuadrature class template defined in GaussJacobiQuadrature.hpp. This class inherits from a base Quadrature class and is responsible for computing quadrature ...

ALG_009 - HOW DOES THE ORTHOGONAL RECURSIVE BISECTION DECIDE WHERE TO CUT THE DOMAIN?

- Verdict: Partial
- Codex reference answer: ORB chooses a cut axis from the domain geometry and then finds a weight-balanced cut/median so recursive splits distribute load more evenly.
- Comparison: The local answer captured the cut-axis choice but did not clearly state that the cut location is chosen from the particle-weight distribution.
- Primary sources: data/raw/ippl/src/Decomposition/OrthogonalRecursiveBisection.h; data/raw/ippl/src/Decomposition/OrthogonalRecursiveBisection.hpp

- Local answer excerpt: Answer: The orthogonal recursive bisection decides where to cut the domain by determining the longest axis of the domain and selecting it for the next cut. This is achieved through the findCutAxis method, which finds the axis with the ...

ALG_010 - HOW DOES INVERSE TRANSFORM SAMPLING GENERATE PARTICLES FROM A GIVEN DISTRIBUTION IN IPPL?

- Verdict: Correct
- Codex reference answer: Inverse transform sampling maps uniform random values in CDF space to samples by estimating the inverse and refining it with Newton-Raphson, dimension by dimension.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Random/InverseTransformSampling.h
- Local answer excerpt: Answer: The InverseTransformSampling class in IPPL generates random samples from a given distribution function by using the inverse of its cumulative distribution function (CDF). The method involves generating uniform random variates a...

ALG_011 - HOW DOES THE FFT CLASS INTERFACE WITH HEFFTE TO PERFORM COMPLEX-TO-COMPLEX AND REAL-TO-COMPLEX TRANSFORMS?

- Verdict: Incorrect
- Codex reference answer: The FFT layer builds heFFTe boxes/plans in setup(), copies IPPL field data into the required views/subviews, calls heFFTe forward/backward transforms, and copies results back for CC/RC/DST/DC variants.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/src/FFT/FFT.hpp
- Local answer excerpt: Answer: The FFT class interfaces with HeFFTe to perform complex-to-complex and real-to-complex transforms through specific methods like transform. The FFT<CCTransform, ComplexField>::transform method performs an in-place FFT on a com...

ALG_012 - HOW DOES IPPL'S PRECONDITIONED CG USE THE PRECONDITIONER DEFINED IN PRECONDITIONER.H?

- Verdict: Partial
- Codex reference answer: PCG applies a chosen preconditioner from Preconditioner.h to the residual, uses the preconditioned residual in its inner products and direction updates, and iterates until tolerance/max iterations.
- Comparison: The local answer recognized the preconditioner family but stayed generic instead of tying it to the actual PCG update flow.
- Primary sources: data/raw/ippl/src/LinearSolvers/Preconditioner.h;
data/raw/ippl/src/LinearSolvers/PCG.h
- Local answer excerpt: Answer: IPPL's preconditioned Conjugate Gradient (PCG) solver uses the preconditioners defined in Preconditioner.h through the setPreconditioner method. This method sets up different preconditioners like Jacobi, Polynomial Newton, Po...

DATA FLOW

FLOW_001 - HOW DOES CHARGE DENSITY FLOW FROM PARTICLES TO THE GRID IN A TYPICAL PIC STEP IN IPPL?

- Verdict: Incorrect
- Codex reference answer: In the PIC workflow, particle charges and positions are deposited onto the grid with CIC (scatter) or FEM assembly (assemble_rhs_from_particles), accumulated into rho, and then normalized/background-shifted in the manager before solving.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/alpine/AlpineManager.h; data/raw/ippl/src/Interpolation/CIC.hpp
- Local answer excerpt: Answer: In IPPL, charge density flows from particles to the grid through a process involving interpolation and scattering operations. The gather method in the ParticleAttrib class is responsible for transferring charge density values...

FLOW_002 - HOW DOES THE ELECTRIC FIELD FLOW FROM THE GRID BACK TO PARTICLES AFTER A POISSON SOLVE?

- Verdict: Incorrect
- Codex reference answer: After the field solve, the manager gathers the electric field back to

particles with CIC gather(...) or FEM interpolate_grad_to_diracs(...), storing particle-local E values for the push step.

- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.

- Primary sources: data/raw/ippl/alpine/AlpineManager.h

- Local answer excerpt: Answer: After solving the Poisson equation to compute the electrostatic potential on the grid, the electric field is computed as the gradient of this potential. This electric field is then transferred back to the particles using interpol...

FLOW_003 - HOW ARE HALO CELLS EXCHANGED BETWEEN NEIGHBORING MPI RANKS FOR A BAREFIELD?

- Verdict: Partial

- Codex reference answer: Halo exchange uses FieldLayout neighbor/range metadata to pack field subviews, send/receive them with MPI, unpack into ghost or physical regions, and handle serial-periodic dimensions locally.

- Comparison: The local answer contains real signal, but it misses an important scope boundary, file, or algorithm detail.

- Primary sources: data/raw/ippl/src/Field/HaloCells.hpp; data/raw/ippl/src/Field/BareField.hpp

- Local answer excerpt: Answer: Halo cells are exchanged between neighboring MPI ranks using the accumulateHalo method of the BareField class. This method checks if there is more than one process involved (layout_m->comm.size() > 1) and, if so, calls hal...

FLOW_004 - HOW ARE PARTICLES MIGRATED BETWEEN MPI RANKS BY PARTICLESPATIALLAYOUT?

- Verdict: Partial

- Codex reference answer: ParticleSpatialLayout::update() applies particle BCs, locates destination ranks, advertises receive counts with RMA, sends all registered attributes, destroys invalid local particles, and receives the incoming ones.

- Comparison: The local answer found the owning routine but skipped most of the send/destroy/receive sequence.

- Primary sources: data/raw/ippl/src/Particle/ParticleSpatialLayout.hpp

- Local answer excerpt: Answer: Particles are migrated between MPI ranks by ParticleSpatialLayout through a combination of methods that manage spatial layouts and communications. The update method triggers particle redistribution based on their new position...

FLOW_005 - HOW DOES FIELDLAYOUT DECIDE WHICH INDEX RANGES LIVE ON WHICH RANK?

- Verdict: Partial

- Codex reference answer: FieldLayout uses the partitioner and the requested parallel/serial decomposition flags to assign each rank a local NDIndex, plus neighbor and communication range tables.

- Comparison: The local answer contains real signal, but it misses an important scope boundary, file, or algorithm detail.

- Primary sources: data/raw/ippl/src/FieldLayout/FieldLayout.h;
data/raw/ippl/src/Partition/Partitioner.h

- Local answer excerpt: Answer: FieldLayout decides which index ranges live on which rank through a combination of methods that calculate domain boundaries and add neighbor information. The findNeighbors method determines neighboring ranks based on the field ...

FLOW_006 - HOW IS A PARAMETERLIST USED TO CONFIGURE THE FFT AND SOLVER CLASSES?

- Verdict: Incorrect

- Codex reference answer: ParameterList is merged into FFT and solver objects, then consumed by setup()/initialize()/solve() for communication mode, pencil/reorder flags, transform direction, solver type, tolerances, preconditioner settings, and output type.

- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.

- Primary sources: data/raw/ippl/src/FFT/FFT.hpp;
data/raw/ippl/src/PoissonSolvers/FFTPeriodicPoissonSolver.h;
data/raw/ippl/src/PoissonSolvers/PoissonCG.h

- Local answer excerpt: Answer: In the IPPL codebase, ParameterList is used to manage and pass configuration parameters for FFT operations. Specifically, a ParameterList is passed as an argument

during the construction of FFT objects (e.g., FFT<RCTransform...

FLOW_007 - HOW DOES DATA FLOW THROUGH THE BASEMANAGER'S PRE_STEP / ADVANCE / POST_STEP HOOKS DURING A SIMULATION?

- Verdict: Correct
- Codex reference answer: BaseManager::run() executes pre_step(), advance(), and post_step() for each timestep; derived managers use those hooks for deposit/solve/gather, time updates, and output.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Manager/BaseManager.h; data/raw/ippl/alpine/AlpineManager.h
- Local answer excerpt: Answer: The BaseManager class defines the pre_step, advance, and post_step methods as virtual functions intended to be overridden by derived classes. These methods are part of the simulation lifecycle management, where pre_step...

FLOW_008 - HOW DOES THE FEM SOLVER MOVE DATA BETWEEN FIELDS AND FEMVECTOR REPRESENTATIONS?

- Verdict: Incorrect
- Codex reference answer: FEM solvers move data by assembling field data into FEM load/stiffness representations (evaluateLoadVector, evaluateAx, FEMVector) and then interpolating the solved field quantities back where needed.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/src/FEM/LagrangeSpace.h; data/raw/ippl/src/FEM/LagrangeSpace.hpp; data/raw/ippl/src/FEM/FEMVector.h
- Local answer excerpt: Answer: The FEM solver uses FEMVector to manage vector operations and halo exchanges. Specifically, methods like accumulateHalo, fillHalo, and deepCopy are used for data movement between fields and FEMVector representations. a...

FLOW_009 - HOW DOES ORTHOGONALRECURSIVEBISECTION REDISTRIBUTE PARTICLES AFTER REBALANCING?

- Verdict: Partial
- Codex reference answer: After ORB recomputes the decomposition, particle ownership changes are realized by updating the layout/regions and then migrating particles with the usual particle update path.
- Comparison: The local answer contains real signal, but it misses an important scope boundary, file, or algorithm detail.
- Primary sources: data/raw/ippl/src/Decomposition/OrthogonalRecursiveBisection.hpp; data/raw/ippl/src/Particle/ParticleSpatialLayout.hpp
- Local answer excerpt: Answer: After rebalancing, OrthogonalRecursiveBisection redistributes particles by performing a binary repartitioning operation. This involves scattering particles (scatterR), recursively bisecting domains based on load balancing cri...

API USAGE

API_001 - HOW DO I CREATE A 3D FIELD OF DOUBLES IN IPPL?

- Verdict: Partial
- Codex reference answer: A typical 3D field is Field<double, 3, UniformCartesian<double, 3>, Cell> constructed from a UniformCartesian mesh and a FieldLayout<3>.
- Comparison: The local answer had the right shape but stayed abstract instead of grounding it in the concrete mesh/layout construction IPPL expects.
- Primary sources: data/raw/ippl/src/Field/Field.h; data/raw/ippl/src/Meshes/UniformCartesian.h; data/raw/ippl/doc/examples/BasicFields.hpp
- Local answer excerpt: Answer: To create a 3D field of doubles in IPPL, you would use the Field class template with appropriate parameters. Based on the provided context, the Field class is defined in Field.h and Field.hpp. The type for double precisio...

API_002 - HOW DO I CONSTRUCT A UNIFORMCARTESIAN MESH WITH CUSTOM SPACING AND ORIGIN?

- Verdict: Partial
- Codex reference answer: Construct the mesh as UniformCartesian<double, Dim> mesh(domain, hx, origin); where hx is the spacing vector and origin is the mesh origin.

- Comparison: The local answer contains real signal, but it misses an important scope boundary, file, or algorithm detail.
- Primary sources: data/raw/ippl/src/Meshes/UniformCartesian.h; data/raw/ippl/doc/examples/BasicMesh.hpp
- Local answer excerpt: Answer: To construct a UniformCartesian mesh with custom spacing and origin, use its constructor defined in UniformCartesian.hpp. The constructor takes parameters for dimensions (NDIndex<Dim>& ndi), mesh spacing (vector_type& hx)...

API_003 - HOW DO I ATTACH A BOUNDARY CONDITION (E.G. PERIODIC) TO A FIELD?

- Verdict: Partial
- Codex reference answer: Create a BConds<FieldT, Dim>, assign per-face BC objects such as PeriodicFace<FieldT>, then call field.setFieldBC(bc).
- Comparison: The local answer contains real signal, but it misses an important scope boundary, file, or algorithm detail.
- Primary sources: data/raw/ippl/src/Field/Field.h; data/raw/ippl/src/Field/BConds.h; data/raw/ippl/src/Field/BcTypes.h
- Local answer excerpt: Answer: To attach a periodic boundary condition to a Field, you can use the setPeriodicBoundaryConditions method provided by classes like FDTDSolverBase. This method sets up periodic boundary conditions for fields by creating and a...

API_004 - HOW DO I DEFINE A CUSTOM PARTICLE CLASS THAT ADDS MASS AND VELOCITY ATTRIBUTES ON TOP OF PARTICLEBASE?

- Verdict: Correct
- Codex reference answer: Define a class derived from ParticleBase<ParticleSpatialLayout<...>>, add mass and velocity attributes as members, and register them with addAttribute() in the constructor.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Particle/ParticleBase.h
- Local answer excerpt: Answer: To define a custom particle class with mass and velocity attributes on top of ParticleBase, you need to derive from ParticleBase and add ParticleAttribute instances for mass and velocity. You should also use the constructor...

API_005 - HOW DO I CALL IPPL::INITIALIZE AND IPPL::FINALIZE IN A MAIN PROGRAM?

- Verdict: Incorrect
- Codex reference answer: Call the free namespace functions ippl::initialize(argc, argv); and ippl::finalize(); from main, as shown in the docs/examples.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/src/Ippl.h; data/raw/ippl/doc/examples/HelloWorld.hpp
- Local answer error: NameError: name 'exact_symbols' is not defined

API_006 - HOW DO I SET UP AND RUN A FORWARD FFT ON AN IPPL FIELD?

- Verdict: Partial
- Codex reference answer: Set up the relevant layouts/fields/parameters, build an FFT<CCTransform, ...> or FFT<RCTransform, ...> object, and call transform(FORWARD, ...) on the field(s).
- Comparison: The local answer contains real signal, but it misses an important scope boundary, file, or algorithm detail.
- Primary sources: data/raw/ippl/src/FFT/FFT.h; data/raw/ippl/src/FFT/FFT.hpp; data/raw/ippl/doc/examples/BasicFFT.hpp
- Local answer error: NameError: name 'exact_symbols' is not defined

API_007 - HOW DO I CONSTRUCT AN FFTPERIODICPOISSONSOLVER AND SOLVE FOR THE POTENTIAL?

- Verdict: Partial
- Codex reference answer: Construct FFTPeriodicPoissonSolver(lhs, rhs) (or default construct then set fields/params), ensure the RHS is attached, and call solve() to write the potential/gradient outputs.
- Comparison: The local answer contains real signal, but it misses an important scope boundary, file, or algorithm detail.

- Primary sources: data/raw/ippl/src/PoissonSolvers/FFTPeriodicPoissonSolver.h; data/raw/ippl/src/PoissonSolvers/FFTPeriodicPoissonSolver.hpp
- Local answer excerpt: Answer: To construct an FFTPeriodicPoissonSolver and solve for the potential, follow these steps: 1. Create instances of FieldLHS and FieldRHS representing the left-hand side (solution field) and right-hand side (source term) of th...

API_008 - HOW DO I INVOKE THE CONJUGATE GRADIENT POISSON SOLVER ON A FIELD?

- Verdict: Incorrect
- Codex reference answer: Use PoissonCG<FieldT> (or the Alpine FieldSolver wrapper around it), set its fields/parameters, and call solve() so the internal CG/PCG object iterates on the field.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/src/PoissonSolvers/PoissonCG.h
- Local answer excerpt: Answer: To invoke the Conjugate Gradient Poisson solver on a Field, you need to create an instance of PoissonCG, set the left-hand side field (the solution field), and call the solve method. Here's how: 1. Create an instance of Pois...

API_009 - HOW DO I SAMPLE PARTICLES FROM A NORMAL DISTRIBUTION USING IPPL RANDOM UTILITIES?

- Verdict: Partial
- Codex reference answer: For Gaussian draws, IPPL exposes both randn convenience generation and the more general InverseTransformSampling + NormalDistribution path.
- Comparison: The local answer contains real signal, but it misses an important scope boundary, file, or algorithm detail.
- Primary sources: data/raw/ippl/src/Random/Randn.h; data/raw/ippl/src/Random/NormalDistribution.h; data/raw/ippl/src/Random/InverseTransformSampling.h
- Local answer excerpt: Answer: The InverseTransformSampling class can be used to generate samples from a specified distribution, including a normal distribution. This class utilizes inverse transform sampling and supports both domain decomposition and non-do...

API_010 - HOW DO I REGISTER PARTICLE ATTRIBUTES WITH PARTICLEBASE SO THAT THEY ARE COMMUNICATED ON MIGRATION?

- Verdict: Correct
- Codex reference answer: Register each particle attribute with addAttribute(...); migration then serializes and transfers all registered attributes automatically.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Particle/ParticleBase.h; data/raw/ippl/src/Particle/ParticleSpatialLayout.hpp
- Local answer excerpt: Answer: To register particle attributes with ParticleBase so that they are communicated on migration, you must define a derived class from ParticleBase and use the addAttribute method to add each attribute. This ensures that the at...

API_011 - HOW DO I ACCESS THE MPI RANK AND SIZE THROUGH THE IPPL COMMUNICATOR?

- Verdict: Correct
- Codex reference answer: Access rank and size with ippl::Comm->rank() and ippl::Comm->size() (or the same methods on a Communicator object).
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Communicate/Communicator.h
- Local answer excerpt: Answer: To access the MPI rank and size through the IPPL Communicator, you can use the methods rank and size respectively. The rank method returns the rank of the current process in the MPI communicator, while the size method ret...

API_012 - HOW DO I USE IPPLTIMINGS TO MEASURE A REGION OF CODE?

- Verdict: Correct
- Codex reference answer: Get a timer handle with IpplTimings::getTimer("name"), surround the code with startTimer() / stopTimer(), and print with IpplTimings::print().
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Utility/IpplTimings.h

- Local answer excerpt: Answer: To measure a region of code using IpplTimings, follow these steps:
1. Create a timer by calling IpplTimings::getTimer("timer name"). This will return a reference to an existing timer or create a new one if it does not exist. ...

PARALLELISM AND KOKKOS

PAR_001 - HOW DOES IPPL ACHIEVE PERFORMANCE PORTABILITY ACROSS CPUS AND GPUS?

- Verdict: Correct
- Codex reference answer: IPPL gets performance portability by combining Kokkos execution/memory abstractions with MPI and heFFTe, so the same code paths can target CPUs and GPUs.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/README.md; data/raw/ippl/src/Utility/ParallelDispatch.h
- Local answer excerpt: Answer: IPPL achieves performance portability across CPUs and GPUs by leveraging Kokkos for parallelism, which provides a programming model that abstracts hardware specifics. The ReflectiveBC::operator() method is an example of how Kok...

PAR_002 - WHY DOES IPPL USE KOKKOS::VIEW FOR FIELD STORAGE?

- Verdict: Partial
- Codex reference answer: IPPL uses Kokkos::View because fields need execution-space-aware storage, subviews, mirrors, and portable kernels across host/device memory spaces.
- Comparison: The local answer did not really explain the motivation; it mostly restated that Views exist.
- Primary sources: data/raw/ippl/src/Field/BareField.h; data/raw/ippl/src/Communicate/Archive.h
- Local answer excerpt: Answer: The retrieved context does not provide a specific explanation for why IPPL uses Kokkos::View for field storage. The chunks cover various aspects of IPPL's functionality and structure but do not explicitly state the reason for c...

PAR_003 - HOW DOES IPPL PARALLELIZE LOOPS OVER FIELD INDICES USING KOKKOS RANGEPOLICY?

- Verdict: Partial
- Codex reference answer: Loops are parallelized through ippl::getRangePolicy / createRangePolicy and the ippl::parallel_for wrapper over Kokkos RangePolicy/MDRangePolicy.
- Comparison: The local answer contains real signal, but it misses an important scope boundary, file, or algorithm detail.
- Primary sources: data/raw/ippl/src/Utility/ParallelDispatch.h
- Local answer excerpt: Answer: IPPL uses RangePolicy from the ParallelDispatch.h file to define policies for parallel execution of loops over field indices. Specifically, RangePolicy is a struct that defines multidimensional range policies using Kokkos's...

PAR_004 - HOW DOES IPPL SUPPORT MIXED PRECISION (FLOAT VS DOUBLE) FOR FIELDS?

- Verdict: Incorrect
- Codex reference answer: Precision is chosen by the field value type (Field<float,...> vs Field<double,...>), and the codebase also contains mixed-precision examples where fields/particles deliberately use different precisions.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/src/Field/Field.h;
data/raw/ippl/alpine/ExamplesWithoutPicManager/LandauDampingMixedPrecision.cpp
- Local answer excerpt: Answer: IPPL supports mixed precision through its unit tests, which include configurations that test components using single (float) and double (double) precision. This is evident from the UNIT_TESTS.md documentation, which lists d...

PAR_005 - HOW DOES IPPL HANDLE MULTIPLE EXECUTION SPACES LIKE OPENMP AND CUDA IN THE SAME BUILD?

- Verdict: Partial
- Codex reference answer: Multiple execution spaces are enabled at build time through IPPL_PLATFORMS and Kokkos/HeFFTe backend flags; IPPL also has container helpers for multi-space data.
- Comparison: The local answer contains real signal, but it misses an important scope boundary, file, or algorithm detail.

- Primary sources: data/raw/ippl/INSTALLATION.md; data/raw/ippl/src/Utility/TypeUtils.h
- Local answer excerpt: Answer: IPPL handles multiple execution spaces like OpenMP and CUDA through the use of Kokkos, which abstracts away the specifics of different backends. The MultispaceContainer class in TypeUtils.h manages elements across different m...

PAR_006 - WHAT IS THE PURPOSE OF PARALLELDISPATCH.H AND WHAT HELPERS DOES IT EXPOSE?

- Verdict: Correct
- Codex reference answer: ParallelDispatch.h wraps Kokkos policies and functors, exposing RangePolicy, getRangePolicy, createRangePolicy, parallel_for, and parallel_reduce.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Utility/ParallelDispatch.h
- Local answer excerpt: Answer: The ParallelDispatch.h file defines abstractions and utilities for parallel execution in the IPPL library, specifically tailored to integrate with Kokkos for handling parallel loops and reductions. It exposes several helpers in...

PAR_007 - HOW DOES THE ARCHIVE CLASS SERIALIZE/DESERIALIZE KOKKOS VIEWS FOR MPI TRANSFER?

- Verdict: Correct
- Codex reference answer: Archive serializes Kokkos views into one-dimensional byte buffers with Kokkos kernels and memcpy, then deserializes them back into views on receive.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/Communicate/Archive.h; data/raw/ippl/src/Communicate/Archive.hpp
- Local answer excerpt: Answer: The Archive class serializes and deserializes Kokkos views by wrapping one-dimensional Kokkos views of type char. Serialization involves copying data from a Kokkos view into this char-based buffer using Kokkos::deep_copy, w...

PAR_008 - HOW ARE MPI TAGS MANAGED IN IPPL TO AVOID COLLISIONS ACROSS SUBSYSTEMS?

- Verdict: Incorrect
- Codex reference answer: MPI tags are managed with fixed base tags in Tags.h and the cyclic tag allocator in TagMaker, which Communicator inherits.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/src/Communicate/Tags.h; data/raw/ippl/src/Communicate/TagMaker.h; data/raw/ippl/src/Communicate/Communicator.h
- Local answer excerpt: Answer: IPPL uses a TagMaker class in the Communicate module to manage MPI tags and avoid collisions across subsystems. The TagMaker class provides methods like next_tag, create_base_tag, and current_tag to generate, create, ...

PAR_009 - WHAT DOES KOKKOS::FENCE DO, AND WHEN DOES IPPL CALL THE WRAPPER IPPL::FENCE?

- Verdict: Partial
- Codex reference answer: Kokkos::fence() waits for outstanding kernels; IPPL exposes ippl::fence() as a runtime wrapper in Ippl.cpp and also fences internally after many kernel/data-movement steps.
- Comparison: The local answer contains real signal, but it misses an important scope boundary, file, or algorithm detail.
- Primary sources: data/raw/ippl/src/Ippl.cpp; data/raw/ippl/src/Field/HaloCells.hpp; data/raw/ippl/src/Communicate/Archive.hpp
- Local answer excerpt: Answer: Kokkos::fence is used to enforce a synchronization barrier, ensuring that all preceding operations are completed before any subsequent operations proceed. In IPPL, ippl::fence acts as a wrapper around Kokkos::fence and is called ...

BOUNDARY AND HALO

BC_001 - WHAT BOUNDARY CONDITION TYPES ARE PROVIDED IN BCTYPES.H?

- Verdict: Correct
- Codex reference answer: BcTypes.h provides periodic, constant, zero, extrapolating, and no-BC face classes.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.

- Primary sources: data/raw/ippl/src/Field/BcTypes.h
- Local answer excerpt: Answer: The BcTypes.h file defines several derived classes representing different boundary condition types: ConstantFace, ExtrapolateFace, NoBcFace, PeriodicFace, and ZeroFace. Evidence: - data/raw/ippl/src/Field/BcTypes.h : ...

BC_002 - HOW DOES A PERIODIC BOUNDARY CONDITION DIFFER FROM A NEUMANN BOUNDARY CONDITION IN IPPL FIELDS?

- Verdict: Partial
- Codex reference answer: Periodic BCs wrap values from the opposite side of the domain; there is no explicit NeumannFace class in this codebase, so the closest non-periodic behavior is constant/extrapolating face treatment.
- Comparison: The local answer's abstention was understandable because the question names a BC type that is not literally implemented, but it still missed the useful source-backed clarification.
- Primary sources: data/raw/ippl/src/Field/BcTypes.h; data/raw/ippl/src/Field/BcTypes.hpp
- Local answer excerpt: Answer: The provided retrieved context does not explicitly describe the differences between periodic and Neumann boundary conditions in IPPL fields. The context only mentions that setPeriodicBoundaryConditions is a method used to set u...

BC_003 - HOW ARE GHOST CELLS POPULATED DURING A HALO EXCHANGE?

- Verdict: Incorrect
- Codex reference answer: Ghost cells are populated mainly by fillHalo() / PeriodicFace::apply(): source slabs are packed, exchanged, and unpacked into ghost regions; accumulateHalo_noghost() is a different operation.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/src/Field/HaloCells.hpp; data/raw/ippl/src/Field/BcTypes.hpp
- Local answer excerpt: Answer: Ghost cells are populated during a halo exchange by calling HaloCells<T, Dim, ViewArgs...>::accumulateHalo_noghost. This method uses exchangeBoundaries with an operation (lhs_plus_assign) to transfer data from halo cells to...

BC_004 - HOW ARE PHYSICAL-CELL VALUES COPIED INTO GHOST CELLS FOR PERIODIC BOUNDARY CONDITIONS?

- Verdict: Incorrect
- Codex reference answer: Periodic ghost cells are filled from opposite-side physical cells in applyPeriodicSerialDim() and the multi-rank PeriodicFace::apply() path.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/src/Field/HaloCells.hpp; data/raw/ippl/src/Field/BcTypes.hpp
- Local answer excerpt: Answer: The assignGhostToPhysical method within the PeriodicFace<Field> class is responsible for copying physical-cell values into ghost cells for periodic boundary conditions. This method calculates appropriate shifts for coordinate...

BC_005 - HOW DOES THE ABSORBING BOUNDARY CONDITION IN THE MAXWELLSOLVERS MODULE DAMP OUTGOING WAVES?

- Verdict: Correct
- Codex reference answer: The absorbing Maxwell BC uses second-order Mur-style formulas with precomputed weights and previous/current/next boundary values plus tangential neighbors to damp outgoing waves.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/MaxwellSolvers/AbsorbingBC.h
- Local answer excerpt: Answer: The second_order_mur_boundary_conditions struct applies second-order Mur absorbing boundary conditions to all boundaries of a 3D computational domain. It uses precomputed weights based on mesh spacing and time step to adjust fi...

NUMERICAL MEANING

NUM_001 - WHAT DOES THE FFT-BASED POISSON SOLVER PHYSICALLY COMPUTE IN TERMS OF COULOMB'S LAW?

- Verdict: Correct
- Codex reference answer: The FFT Poisson solvers compute electrostatic potential (and optionally

field/gradient) from charge density, i.e. the discretized Coulomb potential corresponding to $-\nabla^2 = ?$.

- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/src/PoissonSolvers/FFTOpenPoissonSolver.h; data/raw/ippl/src/PoissonSolvers/FFTPeriodicPoissonSolver.h
- Local answer excerpt: Answer: The retrieved context does not provide specific information about an FFT-based Poisson solver or its relationship to Coulomb's law. The chunks cover various solvers and their methods but do not mention any FFT-based solver that e...

NUM_002 - WHAT IS THE PHYSICAL MEANING OF THE FIELDS ADVANCED BY THE FDTD SOLVER?

- Verdict: Incorrect
- Codex reference answer: The Maxwell solvers evolve electromagnetic field data: electric and magnetic fields (E, B) driven by a source/four-current field.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/src/MaxwellSolvers/Maxwell.h; data/raw/ippl/src/MaxwellSolvers/FDTSolverBase.h
- Local answer excerpt: Answer: The FDTD solver advances the electric (E) and magnetic (B) fields over time. These fields represent electromagnetic wave propagation according to Maxwell's equations. Evidence: - data/raw/ippl/src/MaxwellSolvers/FDTSolverBas...

NUM_007 - WHAT ARE THE NUMERICAL TRADEOFFS BETWEEN THE FFT AND CG POISSON SOLVERS IN IPPL?

- Verdict: Incorrect
- Codex reference answer: FFT Poisson is fast and direct on structured uniform grids with compatible BCs but less flexible; CG/PCG is iterative and may be slower/convergence-limited, but it is more flexible for alternative discretizations, BCs, and preconditioning.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/src/PoissonSolvers/FFTPeriodicPoissonSolver.h; data/raw/ippl/src/PoissonSolvers/PoissonCG.h; data/raw/ippl/src/PoissonSolvers/FEMPoissonSolver.h
- Local answer excerpt: Answer: The FFTPeriodicPoissonSolver in IPPL leverages FFT algorithms for solving periodic Poisson equations, which can be highly efficient due to its parallel nature and low computational complexity. It uses transformations between spat...

BUILD AND INSTALL

BUILD_001 - WHICH THIRD-PARTY LIBRARIES DOES IPPL DEPEND ON, AND WHAT ARE THEY USED FOR?

- Verdict: Incorrect
- Codex reference answer: Core third-party dependencies are MPI for distributed communication, Kokkos for performance portability, heFFTe for FFT backends/plans, FFTW/CuFFT as optional FFT backends, and GoogleTest for tests.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/README.md; data/raw/ippl/INSTALLATION.md; data/raw/ippl/unit_tests/CMakeLists.txt
- Local answer excerpt: Answer: IPPL depends on Kokkos for portable parallelism across CPU/GPU. Specifically, it includes Kokkos_Core.hpp. There is no mention of heFFTe or MPI in the retrieved context. Evidence: - src/Ippl.cpp : Ippl ? contains several utilit...

BUILD_002 - WHAT C++ STANDARD DOES IPPL REQUIRE, AND WHY?

- Verdict: Incorrect
- Codex reference answer: IPPL requires C++20, as stated in the README badge and the installation examples/configuration flags.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/README.md; data/raw/ippl/INSTALLATION.md
- Local answer excerpt: Answer: The retrieved context does not specify which C++ standard IPPL requires or provide any reason for that requirement. Evidence: - data/raw/ippl/src/Ippl.cpp : abort

? The entity is a function named abort defined in the file Ippl...

BUILD_003 - HOW IS IPPL CONFIGURED WITH CMAKE TO ENABLE A GPU BUILD?

- Verdict: Incorrect
- Codex reference answer: GPU builds are enabled through CMake options like IPPL_PLATFORMS=CUDA or IPPL_PLATFORMS="HIP;OPENMP" plus matching Kokkos/architecture/backend flags (Kokkos_ARCH_*, CMAKE_CUDA_ARCHITECTURES, Kokkos_ENABLE_HIP, Heffte_ENABLE_ROCM, etc.).
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/INSTALLATION.md
- Local answer excerpt: Answer: The provided retrieved context does not contain any specific information about configuring IPPL for a GPU build. The documentation and code excerpts focus on general setup, including installation, testing, and compilation options...

BUILD_004 - WHAT OPTIONS DOES INSTALLATION.MD DESCRIBE FOR CONFIGURING KOKKOS FOR IPPL?

- Verdict: Partial
- Codex reference answer: INSTALLATION.md describes Kokkos version selection plus backend/architecture options used with IPPL builds, including CUDA and HIP/OpenMP examples and IPPL_PLATFORMS combinations.
- Comparison: The local answer contains real signal, but it misses an important scope boundary, file, or algorithm detail.
- Primary sources: data/raw/ippl/INSTALLATION.md
- Local answer excerpt: Answer: INSTALLATION.md describes several CMake options related to Kokkos configuration for IPPL. The relevant option is IPPL_PLATFORMS, which can be set to one of SERIAL, OPENMP, CUDA, "OPENMP;CUDA", with a default value of SERIAL...

BUILD_005 - HOW DO I INSTALL IPPL ON THE EULER CLUSTER ACCORDING TO THE DOCUMENTATION?

- Verdict: Correct
- Codex reference answer: The EULER install guide says to connect with VPN/SSH, switch to the new software stack, purge modules, load GCC/CMake/CUDA/OpenMPI, clone the repo, configure a build, compile, and test on an interactive node.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/doc/extras/IPPLonEULER.md
- Local answer excerpt: Answer: To install IPPL on the EULER cluster, follow these steps: 1. Connect to the cluster using SSH and ensure you are connected via the ETH-VPN. 2. Load the new software stack, clean your environment, and load required modules: en...

TESTING AND WORKFLOW

TEST_002 - WHAT NAMING CONVENTIONS DOES WORKFLOW.MD REQUIRE FOR VARIABLES, CONSTANTS, AND MEMBER VARIABLES?

- Verdict: Correct
- Codex reference answer: WORKFLOW.md requires camelCase variables, CAPITAL_CASE compile-time constants, and member names suffixed with _m.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/WORKFLOW.md
- Local answer excerpt: Answer: The naming conventions specified in WORKFLOW.md are as follows: - Variables should use camel case. - Compile-time constants should be in capital letters. - Member variables should end with an underscore followed by 'm'. Evidenc...

TEST_003 - WHEN SHOULD CODE USE KOKKOS::SQRT VERSUS STD::SQRT ACCORDING TO WORKFLOW.MD?

- Verdict: Correct
- Codex reference answer: Use std::sqrt in host-only code, and Kokkos::sqrt in device-capable code such as KOKKOS_INLINE_FUNCTION kernels.
- Comparison: The local answer matches the source-backed reading closely enough to be trusted.
- Primary sources: data/raw/ippl/WORKFLOW.md
- Local answer excerpt: Answer: According to WORKFLOW.md, any instance of a mathematical function in

host-only code should use symbols from the standard library, e.g., `std::sqrt`. Any instances occurring in device-code or code that might be run on a device (su...

TEST_004 - HOW DOES IPPL PROFILE MPI CALLS USING MPIP?

- Verdict: Incorrect
- Codex reference answer: IPPL profiles MPI calls with mpiP by preloading libmpiP.so, optionally setting MPIP flags, then parsing the produced .mpiP output with the bundled scripts/mpiP.py.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/README.md; data/raw/ippl/scripts/mpiP.py
- Local answer excerpt: Answer: The provided context does not mention IPPL profiling MPI calls with mpiP. The chunks cover initialization, configuration, synchronization, and error handling functions in Ippl.cpp, declarations of utility classes and inline poi...

TEST_005 - WHAT DOES THE SLURM JOB TEMPLATE FOR GWENDOLEN IN THE README LOOK LIKE?

- Verdict: Incorrect
- Codex reference answer: The README does include a Gwendolen SLURM template showing the gmerlin6 cluster, gwendolen partition, GPU count, and `srun ./<your_executable> <args> --kokkos-map-device-id-by=mpi_rank`.
- Comparison: The local answer either missed the relevant source-backed answer or contradicted the code/docs materially.
- Primary sources: data/raw/ippl/README.md
- Local answer excerpt: Answer: The provided context does not contain a specific SLURM job template for Gwendolen. The README.md mentions example job scripts for running on the local PSI computing cluster but does not provide a detailed template for Gwendolen. ...

CONCLUSION

This V2 run is a cleaner measurement than the first round because the questions are now restricted to what should be answerable from src/ and repository documentation. The local system is clearly useful on structural questions, but it still loses ground when the answer depends on retrieving the right implementation file or synthesizing across a few related source files.

The best next step is still retrieval quality: make it more likely to surface the exact solver, particle, interpolation, or documentation files that match the query intent, then keep the final prompt focused instead of letting one nearby file dominate the context.